Roteiro do vídeo — Tech Challenge Fase 2

ToggleMaster: de monolito a microsserviços no Amazon EKS · versão para validação

Parte	Conteúdo	Tempo
Parte 1	Abertura, arquitetura, Docker e ambiente local, infraestrutura AWS e os 3 data stores	≈ 9 min
Parte 2	Kubernetes, segredos, Ingress, escalabilidade (HPA), prova no DynamoDB e encerramento	≈ 10 min
Total	Limite do enunciado: 20 minutos	≈ 19 min

A apresentação é feita por dois integrantes, um em cada parte. Trechos marcados com **[opcional]** podem ser cortados se o tempo apertar.

Confirmar antes de gravar
1) Quantas instâncias RDS ficaram no ambiente final (o roteiro considera três, uma por serviço). 2) Os parâmetros do endpoint /evaluate (o roteiro usa user_id e flag_name). 3) Que nenhuma credencial aparece na tela: .env.aws, AWS Details, valores do Secrets Manager e a MASTER_KEY.

Checklist do enunciado

O vídeo precisa mostrar	Onde
docker-compose com os 9 containers	Parte 1
Cluster e recursos AWS	Parte 1
Explicação dos 3 data stores	Parte 1
Pods rodando e Ingress funcionando	Parte 2
HPA escalando sob carga	Parte 2
Dados gravados no DynamoDB	Parte 2

Parte 1 — Arquitetura, ambiente local e AWS

1A · Abertura e arquitetura · 2 min

Deixe pronto antes de gravar
Slide de abertura e diagrama de arquitetura.

Tempo	Na tela	Fala
0:00	Slide de título	Olá, somos o grupo [nome] e vamos apresentar a Fase 2 do Tech Challenge: a evolução do ToggleMaster de monolito para microsserviços, rodando no Amazon EKS.
0:20	Diagrama	Na Fase 1, o ToggleMaster era uma única aplicação numa EC2. Agora ele foi dividido em cinco microsserviços. O auth-service, em Go, cria e valida as chaves de API. O flag-service e o targeting-service, em Python, cuidam das flags e das regras de segmentação. O evaluation-service, em Go, é o caminho crítico: responde se uma flag está ligada para um usuário. E o analytics-service é um worker que consome eventos de uma fila e grava os dados de uso.
1:20	Seta evaluation → SQS → analytics	Um ponto importante é o desacoplamento: o evaluation publica eventos numa fila SQS e o analytics consome no seu próprio ritmo. Se o analytics ficar lento, a avaliação das flags não é afetada.

1B · Conteinerização e ambiente local · 2 min 30 s

Deixe pronto antes de gravar
VS Code com um Dockerfile Go aberto e o Ubuntu na pasta do projeto, com o docker compose já rodando.

Tempo	Na tela	Fala
2:00	Dockerfile do auth-service	Cada serviço foi conteinerizado com Dockerfile multi-stage. No primeiro estágio, compilamos o binário Go. No segundo, copiamos só o binário para uma imagem Alpine mínima, que roda com um usuário sem privilégio de root.
2:25	docker images	O resultado aparece no tamanho: as imagens Go têm cerca de 30 MB, enquanto as Python ficam em torno de 200 MB, porque carregam o interpretador e as bibliotecas.
2:45	Manual técnico, seção de problemas	**[opcional]** No caminho, encontramos erros no próprio código fornecido: o go.mod do auth declarava um pacote como módulo, havia imports sem uso, que em Go impedem a compilação, e o evaluation importava um pacote errado. Também tratamos problemas de ambiente, como o final de linha do Windows e o proxy corporativo. Tudo foi corrigido e documentado.
3:20	docker compose ps	Antes de ir para a nuvem, validamos tudo localmente com o Docker Compose. São nove containers: os cinco serviços e quatro bancos, sendo dois PostgreSQL, um Redis e um DynamoDB Local.
3:50	Health checks 8001 a 8005	Os cinco serviços respondem no health check. Com o ambiente local validado, partimos para a AWS.

Comandos desta parte:

```
docker images | grep togglemaster
docker compose ps
for p in 8001 8002 8003 8004 8005; do echo -n "$p: "; curl -s localhost:$p/health; echo; done
```

1C · Infraestrutura AWS e data stores · 4 min 30 s

Deixe pronto antes de gravar
Abas do console abertas: EKS, ECR, RDS, ElastiCache, DynamoDB, SQS e Security Groups.

Tempo	Na tela	Fala
4:30	EKS: cluster	A infraestrutura está no AWS Academy, na região us-east-1. O núcleo é o cluster Amazon EKS, eks-togglemaster, que gerencia o plano de controle do Kubernetes.
4:50	Compute: node group	Os workloads rodam num node group com duas instâncias t3.medium, usando a LabRole. Começamos com t3.small, mas trocamos: o t3.small comporta só onze pods por nó, e precisávamos de espaço para o autoscaling.
5:20	ECR	As imagens ficam no Amazon ECR, num repositório privado por serviço. O evaluation está na versão 3, que ganhou suporte a Redis com criptografia em trânsito.
5:45	RDS	Agora os três tipos de armazenamento, cada um escolhido pelo tipo de dado. O primeiro é o PostgreSQL no RDS, para o auth, o flag e o targeting. São dados relacionais que exigem consistência: uma chave de API ou uma regra não pode se perder nem ficar inconsistente. Cada serviço tem seu próprio banco, seguindo o padrão database-per-service.
6:30	ElastiCache	O segundo é o Redis no ElastiCache, usado pelo evaluation. Como esse serviço é chamado a cada requisição, ele não pode consultar o banco toda vez. O Redis guarda as flags em memória e responde em milissegundos.
7:00	DynamoDB	O terceiro é o DynamoDB, para o analytics. Cada avaliação vira um evento, então o volume é alto e cresce rápido. O DynamoDB é NoSQL, escala automaticamente e cobra por requisição, o que se encaixa nesse padrão de muita escrita.
7:35	SQS	Entre o evaluation e o analytics está a fila SQS, evaluation-analytics-queue, que desacopla os dois serviços.
7:55	Security Groups	Nenhum banco é público. O RDS e o ElastiCache ficam em sub-redes privadas e só aceitam conexão vinda do security group do cluster, nas portas 5432 e 6379.
8:20	Add-ons: Metrics Server	**[opcional]** Também instalamos o Metrics Server, que fornece as métricas de CPU para o autoscaling. Ele teve um conflito de configuração na atualização, resolvido passando a gestão do componente para o add-on do EKS.
8:40	Diagrama	Com a infraestrutura pronta, vamos ver como os serviços rodam dentro do cluster.

Parte 2 — Kubernetes, escalabilidade e encerramento

2A · Manifestos, segredos e Ingress · 5 min

Deixe pronto antes de gravar
Pods Running, hop limit 2 conferido nos nós, CloudShell na pasta dos manifestos e a variável LB preenchida. MASTER_KEY já numa variável, sem aparecer na tela.

Tempo	Na tela	Fala
9:00	Pastas + kustomization.yaml	Os manifestos estão num repositório de infraestrutura separado do código, organizados por serviço e aplicados com Kustomize, num único comando.
9:20	kubectl get ns	Cada serviço tem seu próprio namespace. Isso isola os recursos, e os serviços conversam pelo DNS interno do cluster, como flag-service.flag.svc.cluster.local.
9:45	Deployment do flag-service	Destaco três pontos deste Deployment. Primeiro, as probes de startup, readiness e liveness, para o Kubernetes saber quando o pod está pronto e reiniciá-lo se travar. Segundo, os requests e limits de CPU e memória, que o autoscaling usa como referência. Terceiro, a segurança: o container roda sem root, com sistema de arquivos somente leitura e sem capabilities extras.
10:40	ExternalSecret (só nomes)	Nenhuma credencial está nos arquivos. Senhas e URLs dos bancos ficam no AWS Secrets Manager, e o External Secrets Operator sincroniza esses valores para Secrets do Kubernetes. Os pods acessam a AWS pela LabRole do nó, sem chaves salvas.
11:20	get externalsecret / get jobs	Os quatro segredos estão sincronizados. Para criar as tabelas, usamos Jobs que executam os scripts SQL de cada serviço no RDS, substituindo o passo manual da Fase 1.
11:45	get pods,svc	Os cinco serviços estão Running, cada um exposto internamente por um Service do tipo ClusterIP.
12:05	get ingress	Para entrada externa, usamos o Ingress NGINX com as rotas /auth, /flags, /rules e /evaluate, atrás de um único Load Balancer. O analytics não tem rota, porque é um worker interno. Registramos que o ingress-nginx da comunidade foi aposentado em março de 2026; em produção, migraríamos para Traefik ou Gateway API.
12:45	Chamadas pelo Ingress	Agora o fluxo completo pelo Load Balancer: criamos uma chave no auth, cadastramos uma flag e consultamos o evaluation, que responde se ela está ativa.
13:30	Resposta do /evaluate	**[opcional]** Entre os desafios desta etapa, aplicamos recursos por engano num cluster local em vez do EKS, e descobrimos que os pods precisavam do hop limit 2 no serviço de metadados para usar a LabRole.

Comandos desta parte:

```
kubectl config current-context
kubectl get ns
kubectl get externalsecret -A
kubectl get jobs -A
kubectl get pods,svc -A | grep -E "auth|flag|targeting|evaluation|analytics"
kubectl get ingress -A
LB=$(kubectl get ingress -A -o jsonpath='{.items[0].status.loadBalancer.ingress[0].hostname}')
KEY=$(curl -s -X POST http://$LB/auth/admin/keys -H "Authorization: Bearer $MASTER_KEY" \
  -H "Content-Type: application/json" -d '{"name":"demo"}' | grep -o 'tm_key_[^"]*')
curl -s -X POST http://$LB/flags -H "Authorization: Bearer $KEY" -H "Content-Type: application/json" \
  -d '{"name":"enable-new-dashboard","description":"demo","is_enabled":true}'
curl -s "http://$LB/evaluate?user_id=user-123&flag_name=enable-new-dashboard"
```

2B · Escalabilidade, DynamoDB e encerramento · 5 min

> **Deixe pronto antes de gravar**
> Três janelas do CloudShell lado a lado (carga, HPA e pods). Ensaiar a carga antes para saber quanto tempo leva para escalar e cortar a espera na edição.

Tempo	Na tela	Fala
14:00	kubectl top nodes	Agora a escalabilidade. O Metrics Server coleta o uso de CPU e memória de nós e pods, e essa informação alimenta o autoscaling.
14:20	kubectl get hpa -A	Configuramos o Horizontal Pod Autoscaler para o evaluation e o analytics, os serviços com maior variação de carga. Se o uso médio de CPU passar de 30% do que o pod solicita, o Kubernetes cria réplicas, até cinco.
14:50	Janela 1: carga	Vamos gerar carga contínua contra o evaluation, simulando muitos clientes consultando flags ao mesmo tempo.
15:10	Janelas 2 e 3	Acompanhem a coluna de CPU subindo no HPA. Quando passa do alvo, novas réplicas do evaluation aparecem e entram em execução.
16:00	Réplicas Running	O evaluation saiu de uma para [N] réplicas, sem intervenção manual, e o Service distribui as requisições entre elas.
16:30	Carga parada	Ao encerrar a carga, o HPA aguarda uma janela de estabilização e reduz as réplicas. Só pagamos pela capacidade enquanto ela é necessária.
17:00	DynamoDB: itens	E aqui a prova de ponta a ponta: cada avaliação gerou um evento na fila SQS, o analytics consumiu e gravou no DynamoDB. Estes são os registros criados durante o teste.
17:30	Slide: lições aprendidas	Entre as lições: validar o ambiente antes de escrever código, testar localmente antes de ir para a nuvem, manter credenciais fora dos arquivos e sempre conferir o contexto do kubectl antes de aplicar qualquer coisa.
18:10	Slide: próximos passos	**[opcional]** Como próximos passos: pipeline de CI/CD para build e deploy automáticos, migração do Ingress para Gateway API e launch template para fixar a configuração dos nós.
18:35	Slide final	Obrigado. Este foi o ToggleMaster na Fase 2, apresentado pelo grupo [nome].

Comandos desta parte:

```
kubectl top nodes
kubectl get hpa -A
# Janela 1 - carga (abrir mais janelas com carga2, carga3 se a CPU nao passar de 30%)
kubectl run carga --rm -it --image=busybox --restart=Never -- /bin/sh -c \
 'while true; do wget -q -O- "http://evaluation-service.evaluation.svc.cluster.local:8004/evaluate?user_id=user-123&fla
# Janelas 2 e 3
kubectl get hpa -n evaluation -w
kubectl get pods -n evaluation -w
# Prova no DynamoDB
aws dynamodb scan --table-name ToggleMasterAnalytics --max-items 5 --output table
```

Dicas de gravação

- Gravar cada parte separadamente e juntar na edição: um erro não obriga a refazer o vídeo inteiro.
- Aumentar a fonte do terminal e deixar os comandos prontos para colar.
- Cortar na edição os momentos de espera (pods subindo, HPA escalando).
- Ensaiar a frase de passagem entre as partes.
- Conferir a duração final: o limite é de 20 minutos.